C++ PROGRAMMING TEXTBOOK NOTES

Module: Pure Virtual Functions & Abstract Classes

Author: Gaurav Kandpal | **Platform:** CS-Simplified | **Level:** Beginner to Advanced

1. WHAT IS A PURE VIRTUAL FUNCTION?

A **Pure Virtual Function** in C++ is a virtual member function declared in a base class that has **no implementation** within that base class. It is defined by appending = 0 to its function signature inside the class definition.

Any class containing at least one pure virtual function automatically becomes an **Abstract Base Class (ABC)**. Abstract classes serve purely as architectural contracts/interfaces and cannot be instantiated directly.

Fundamental Syntax:

```
class BaseClass {
public:
    virtual return_type function_name(parameter_list) = 0; // Pure Virtual Function
};
```

2. KEY PROPERTIES & RULES OF ABSTRACT CLASSES

- **No Direct Instantiation:** You cannot create direct objects of an abstract base class (e.g., A obj; triggers a compile-time error).
- **Mandatory Overriding:** Derived classes *must* provide concrete implementations for all pure virtual functions, unless the derived class itself is intended to be abstract.
- **Polymorphic Pointers & References:** Although direct objects cannot be created, you *can* declare pointers and references of the Abstract Base Class type (e.g., A* ptr = new B();).
- **Abstract Class Destructors:** Base abstract classes must always specify a **virtual destructor** to guarantee safe dynamic deallocation of derived instances.

3. PRACTICAL C++ IMPLEMENTATION

```cpp
// Enhanced Pure Virtual Function Implementation
#include <iostream>
using namespace std;

// Abstract Base Class (Cannot be instantiated)
class ClassA {
public:
    // Pure Virtual Function declaration
    virtual void fun1() = 0;

    // Virtual Destructor ensuring clean resource cleanup
    virtual ~ClassA() {}
};

// Derived Class implementing the contract
class ClassB : public ClassA {
public:
    // Overriding the pure virtual function
    void fun1() override {
        cout << "Hello jii Sita ram!!\n";
    }
};

int main() {
    // ClassA objA; // COMPILE ERROR: Cannot instantiate abstract class!

    // Method 1: Stack Allocation of Concrete Derived Object
    ClassB objB;
    objB.fun1(); // Output: Hello jii Sita ram!!

    // Method 2: Polymorphic Pointer Dispatch
    ClassA* ptr = new ClassB();
    ptr->fun1(); // Dynamic resolution calls ClassB::fun1()

    delete ptr; // Clean deallocation
    return 0;
}
```

4. COMPARISON: VIRTUAL VS. PURE VIRTUAL FUNCTIONS

Feature / Property	Standard Virtual Function	Pure Virtual Function
Declaration Syntax	virtual void show();	virtual void show() = 0;
Base Class Implementation	Must provide a default implementation.	Usually none (acts purely as interface).
Derived Class Requirement	Optional to override (inherits base logic).	**Mandatory to override** to make class concrete.
Class Instantiation	Base class can be instantiated.	Base class becomes **Abstract** (Cannot instantiate).
Design Intent	Default behavior with optional customization.	Strict architectural interface enforcement.

5. REAL-WORLD ARCHITECTURAL ANALOGY

REAL-WORLD ANALOGY: Hardware Plugin Interface Standard

• **Abstract Base Class (USB Interface Standard):** Defines that any USB device *must* support connect() and transferData() without defining how mouse, keyboard, or flash drive circuits work internally.
• **Pure Virtual Function (virtual void connect() = 0;):** The contract forcing manufacturers to implement connection logic.
• **Concrete Derived Classes (Mouse / Keyboard / Disk):** Provide actual internal implementations.
• **Why it Matters:** Computers interact with any USB device using the abstract USB interface pointer without needing to know specific device hardware details.